HDOC Day-63

Finding the Missing Number in an Array Using Functions and Pointers in C

Aim

To write a C program using a variable-length integer array, a user-defined function, pointers, and arithmetic operations to find the missing number from distinct integers in the range 0 to size.

Operations

1. Read the size and elements of a variable-length integer array nums[].
2. Accept exactly size distinct integers from 0 to size, with one number missing.
3. Pass the array to missing_number() using its address.
4. Find and display the missing integer using the expected and actual sums.
5. Use pointer arithmetic to access array elements.

Algorithm

1. Start the program and read size.
2. If size is not positive, display an invalid-size message and stop.
3. Declare nums[size] and read size distinct elements.
4. Pass nums and size to missing_number().
5. Calculate expectedSum = size * (size + 1) / 2.
6. Initialize actualSum to zero and add all array elements using pointer arithmetic.
7. Calculate missing = expectedSum - actualSum.
8. Display the missing number and stop.

C Program

```
#include <stdio.h>

void missing_number(int *nums, int size);

int main()
{
    int size, i;

    printf("Enter the size of array: ");
    scanf("%d", &size);

    if (size <= 0)
    {
        printf("Invalid array size!\n");
        return 0;
    }

    int nums[size];

    printf("Enter %d distinct array elements: ", size);
    for (i = 0; i < size; i++)
    {
        scanf("%d", &nums[i]);
    }

    missing_number(nums, size);
    return 0;
}

/* Function to find the missing number */
void missing_number(int *nums, int size)
{
    int i;
    long long expectedSum, actualSum = 0;
    long long missing;

    /* Expected sum of integers from 0 to size */
    expectedSum = (long long)size * (size + 1) / 2;

    for (i = 0; i < size; i++)
    {
        actualSum += *(nums + i);
    }

    missing = expectedSum - actualSum;
    printf("\nMissing Number: %lld\n", missing);
}
```

Explanation

The function missing_number(int *nums, int size) receives the address of the first array element. The expression *(nums + i) accesses nums[i]. Since the input contains every number from 0 through size except one, subtracting the actual array sum from the expected sum identifies the missing number.

```
Expected Sum = size * (size + 1) / 2
Missing Number = Expected Sum - Actual Sum
```

Worked Example

For nums = 3 0 1 and size = 3, the expected sum is 3 * 4 / 2 = 6, while the actual sum is 3 + 0 + 1 = 4. Therefore, the missing number is 2.

Test Case Table

Test	Input Array	Size	Expected Result
1	3 0 1	3	2
2	0 1	2	2
3	9 6 4 2 3 5 7 0 1	9	8
4	0 1 2 3 5	5	4
5	1 2 3 4	4	0
6	0 1 2 3	4	4
7	1	1	0
8	0	1	1
9	2 0 3 4	4	1
10	size = 0	0	Invalid array size

Sample Outputs

Missing Number in the Middle

```
Enter the size of array: 3
Enter 3 distinct array elements: 3 0 1

Missing Number: 2
```

Missing Number at the End

```
Enter the size of array: 4
Enter 4 distinct array elements: 0 1 2 3

Missing Number: 4
```

Missing Number at the Beginning

```
Enter the size of array: 4
Enter 4 distinct array elements: 1 2 3 4

Missing Number: 0
```

Larger Array

```
Enter the size of array: 9
Enter 9 distinct array elements: 9 6 4 2 3 5 7 0 1

Missing Number: 8
```

Invalid Array Size

```
Enter the size of array: 0
Invalid array size!
```

Result

The program finds the missing integer in the range 0 to size using a variable-length array, pointers, a user-defined function, and a single summation loop. It handles a missing value at the beginning, middle, or end of the range in O(n) time and O(1) auxiliary space within the function.